5-Minute Presentation Script

Two-Person Format | AISC Project Presentation

Timing Guide

Slide 1: 0:00 - 0:25
Slide 2: 0:25 - 1:10
Slide 3: 1:10 - 2:00
Slide 4: 2:00 - 2:50
Slide 5: 2:50 - 3:45
Slide 6: 3:45 - 4:30
Slide 7: 4:30 - 5:00

SLIDE 1 - Title Slide

Person A: Good morning, everyone. We are presenting our project titled 'AI-Powered Device Troubleshooting Assistant using the AO Star Algorithm on AND-OR Graphs.'

Person B: My name is [Name], and this is [Name]. Our project applies a classical AI search technique to a very practical everyday problem: figuring out why your device broke - and fixing it as cheaply as possible.

SLIDE 2 - The Problem

Person A: Think about the last time your laptop refused to start. You probably tried a few things at random - maybe you charged it, restarted it, or panicked. That random approach is the core problem we are solving. One symptom like 'laptop won't start' can have dozens of causes: a dead battery, a corrupted OS, a loose RAM stick, or a failed screen. A static discharge reset costs nothing and takes two minutes; replacing the motherboard costs $280.

Person B: Choosing the expensive fix first when a free one would have worked wastes money. Our goal was to build a system that always recommends the minimum-cost repair path first and automatically replans if a step does not work.

SLIDE 3 - AI Approach: AND-OR Graphs

Person B: We modelled each device fault as an AND-OR graph. An OR node means the fault can be caused by ANY ONE of its children - you fix whichever is the actual cause. An AND node means ALL children must be addressed. 'Laptop Won't Start' is an OR node - it could be a software issue, a power issue, a display issue, or a hardware issue.

Person A: Each category is itself an OR node with concrete repair actions as leaf nodes. 'Software Issue' has: boot in safe mode ($0), run startup repair ($0), or reinstall OS ($0). 'Hardware Issue' has repairs from $0 (reseat RAM) to $280 (replace motherboard). Our Printer problem uses AND nodes where two steps are both mandatory.

SLIDE 4 - AO* Algorithm

Person A: We search this AND-OR graph using the AO Star algorithm. AO Star assigns each node a cost estimate f(n). For leaf nodes, f equals the actual repair cost. For OR nodes, f equals the minimum cost among children. For AND nodes, f equals the sum of all children's costs. The algorithm computes these bottom-up: children first, parents after.

Person B: When the user tries a fix and it fails, we mark that leaf with infinite cost and propagate the change upward in O(depth) time. The OR parent automatically picks its next-best child. This is replanning - instant, with no full restart needed.

SLIDE 5 - Live Example Trace

Person B: The root is 'Laptop Won't Start.' AO Star recommends 'Boot in Safe Mode' - cost $0. If the user says it did not work, we mark it failed. Now 'Run Startup Repair' - also $0 - is recommended. If that fails too, and Reinstall OS also fails, the entire Software category is marked failed.

Person A: AO Star then moves to the Power category: 'Static Discharge Reset' - still $0. Only after exhausting all free options does it suggest paid repairs: Replace Charger ($15), then Replace Battery ($50). The system always guides from free to expensive - never the other way around.

SLIDE 6 - Results and Comparison

Person A: We ran a Monte Carlo simulation with 2,000 trials to measure how much AO Star saves. On 'Laptop Won't Start', random users spent an average of $267 before finding the correct fix. The worst case was $535. AO Star's path starts at $0.

Person B: Across all four device categories - laptop, WiFi, phone overheating, and printer - AO Star consistently delivers the minimum-cost fix path first. The simulation shows an average saving of over 90% and 200+ minutes compared to random trial-and-error.

SLIDE 7 - Conclusion

Person B: To summarise: we built a complete AI troubleshooting assistant with an AND-OR graph knowledge base covering four device types, a correct AO Star implementation with dynamic replanning, an interactive CLI session, and a statistical comparison module.

Person A: Future work includes adding success probabilities for a probabilistic model, a web frontend, and learning heuristics from repair history. AO Star is a 60-year-old algorithm - but it solves a very modern, very real problem better than most users solve it on their own. Thank you.

Both: We welcome your questions.

Speaker Notes

Delivery tips

Speak at 140-150 words per minute. Each slide is approximately 40 seconds for Person A and 40 seconds for Person B. Stay within your slot.

Transitions

Person A ends each slide on a sentence that naturally hands to Person B, and vice versa. Practice transitions so they feel smooth, not mechanical.

Slide 5 - Live Example

If you have a terminal open, run: python run.py -> choose option 1 -> problem 1. Show the AO* plan on screen while narrating the trace.

Slide 6 - Results

Run option 3 -> problem 1 to show the live Monte Carlo comparison. The numbers in the script match the simulation output exactly.

Handling questions

Algorithm questions -> Person B. Knowledge base / results -> Person A. For anything else, refer to the viva document (100 Q&A).

Key numbers to memorise

$0 = AO* start cost | $267 avg random | $535 random worst | 90%+ savings | O(depth) replan | 2000 Monte Carlo trials